

Exercises 1

1. A server does not reply only to one client and exit. It waits indefinitely, replying to all clients until it is forced to terminate by the host computer. Modify the server to do this.
2. An echo server is a simple server that receives a message from a client and sends the same message back to the client. Write a simple echo server which achieves this task.
3. Write a server that uses the hypothetical *Capitalizer Service Protocol (CSP)* to provide a capitalizer service. CSP works as follows:
 - First the client initializes the connection by sending an integer which represents the number of sentences it is going to send. (Let's say the number is *n*) the CSP server sends the characters 'ack' back to the client, acknowledging the message.
 - CSP server then sits in a while loop that iterates '*n*' times. At each iteration, the server does the following:
 - receive a sentence from the client.
 - Capitalize it
 - send it back to the client.
4. Write a time server that, when connected, sits in a loop and sends the current time each second to the receiver.

(1) 1) Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 3456
#define BUFFER_SIZE 1024
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    socklen_t addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];
    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    // Bind socket
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 5) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("Persistent server running...\n");
    while (1) {
        // Accept new connection
        new_socket = accept(server_fd, (struct sockaddr *)&address, &addrlen);
        if (new_socket < 0) {
            perror("Accept failed");
        }
    }
}
```

```

// Accept new connection
new_socket = accept(server_fd, (struct sockaddr *)&address, &addrlen);
if (new_socket < 0) {
    perror("Accept failed");
    continue;
}

printf("New client connected!\n");

// Fork a new process to handle client
if (fork() == 0) {
    close(server_fd); // Close listening socket in child process
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int bytes_received = recv(new_socket, buffer, BUFFER_SIZE, 0);
        if (bytes_received <= 0) {
            printf("Client disconnected.\n");
            close(new_socket);
            exit(0);
        }
        printf("Received from client: %s\n", buffer);
        send(new_socket, buffer, strlen(buffer), 0);
    }
}
close(new_socket); // Parent closes client socket after forking
}
return 0;

```

```

hirushi@HS:~/Lab06$ vim 1_s.c
hirushi@HS:~/Lab06$ gcc -o 1_s 1_s.c
hirushi@HS:~/Lab06$ ./1_s
Persistent server running...
New client connected!
Received from client: helloooo nice to meet you
Client disconnected.

```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 3456
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    struct sockaddr_in servaddr;
    char buffer[BUFFER_SIZE];
    char message[BUFFER_SIZE];

    // Create socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1"); // Change if server is remote
    // Connect to server
    if (connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }
    printf("Connected to the server. Type messages (type 'exit' to quit):\n");
    while (1) {
        printf("You: ");
        fgets(message, BUFFER_SIZE, stdin);
        message[strcspn(message, "\n")] = '\0'; // Remove newline

        if (strcmp(message, "exit") == 0) {
            printf("Closing connection...\n");
            break;
        }

        // Send message to server
        if (write(sockfd, message, strlen(message)) < 0) {
            perror("Write failed");
            exit(EXIT_FAILURE);
        }

        // Receive response from server
        if (read(sockfd, buffer, BUFFER_SIZE) < 0) {
            perror("Read failed");
            exit(EXIT_FAILURE);
        }

        printf("Server: %s\n", buffer);
    }
}
```

-- INSERT --

1,1

```
hirushi@HS:~$ cd C0223
hirushi@HS:~/C0223$ cd
hirushi@HS:~$ cd Lab06
hirushi@HS:~/Lab06$ vim 1_c.c
hirushi@HS:~/Lab06$ gcc -o 1_c 1_c.c
hirushi@HS:~/Lab06$ ./1_c
Connected to the server. Type messages (type 'exit' to quit):
You: helloooo nice to meet you
Server: helloooo nice to meet you
You: exit
Closing connection...
hirushi@HS:~/Lab06$
```

2)server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 3579
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }

    // Configure server address structure
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Bind socket to port
    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // Listen for connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
}
```

```

    exit(EXIT_FAILURE);
}

printf("Echo server listening on port %d...\n", PORT);

// Accept client connection
if ((new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen)) < 0) {
    perror("Accept failed");
    exit(EXIT_FAILURE);
}

printf("Client connected.\n");

// Echo loop
while (1) {
    memset(buffer, 0, BUFFER_SIZE);
    int bytes_read = read(new_socket, buffer, BUFFER_SIZE);
    if (bytes_read <= 0) {
        printf("Client disconnected.\n");
        break;
    }

    printf("Received: %s", buffer);

    // Send back the same message
    send(new_socket, buffer, bytes_read, 0);
}

close(new_socket);
close(server_fd);
return 0;

```

```

hirushi@HS:~$ cd Lab06
hirushi@HS:~/Lab06$ vim 2_s.c
hirushi@HS:~/Lab06$ gcc -o 2_s 2_s.c
hirushi@HS:~/Lab06$ ./2_s
Echo server listening on port 3579...
Client connected.
Received: hellooo . I'm sorry
Received: What are u doing now

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 3579
#define BUFFER_SIZE 1024

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};

    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        exit(EXIT_FAILURE);
    }

    // Configure server address
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IP address from text to binary
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address / Address not supported");
        exit(EXIT_FAILURE);
    }

    // Connect to server
    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }

```

```

printf("Connected to server. Type messages to send:\n");

while (1) {
    printf("> ");
    fgets(buffer, BUFFER_SIZE, stdin);

    if (strcmp(buffer, "exit\n") == 0) {
        printf("Closing connection.\n");
        break;
    }

    // Send message
    send(sock, buffer, strlen(buffer), 0);

    // Receive echo
    memset(buffer, 0, BUFFER_SIZE);
    int bytes_read = read(sock, buffer, BUFFER_SIZE);
    printf("Echo from server: %s", buffer);
}

close(sock);
return 0;

```

```
hirushi@HS:~$ cd Lab06
hirushi@HS:~/Lab06$ vim 2_c.c
hirushi@HS:~/Lab06$ gcc -o 2_c 2_c.c
hirushi@HS:~/Lab06$ ./2_c
Connected to server. Type messages to send:
> hellooo . I'm sorry
Echo from server: hellooo . I'm sorry
> What are u doing now
Echo from server: What are u doing now
> █
```

3) Server

```
    exit(EXIT_FAILURE);
}

printf("Echo server listening on port %d...\n", PORT);

// Accept client connection
if ((new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen)) < 0) {
    perror("Accept failed");
    exit(EXIT_FAILURE);
}

printf("Client connected.\n");

// Echo loop
while (1) {
    memset(buffer, 0, BUFFER_SIZE);
    int bytes_read = read(new_socket, buffer, BUFFER_SIZE);
    if (bytes_read <= 0) {
        printf("Client disconnected.\n");
        break;
    }

    printf("Received: %s", buffer);

    // Send back the same message
    send(new_socket, buffer, bytes_read, 0);
}

close(new_socket);
close(server_fd);
return 0;
```

```
##include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <arpa/inet.h>
#define PORT 3109
#define BUFFER_SIZE 1024

// Function to capitalize a string
void capitalize(char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = toupper(str[i]);
    }
}

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }

    // Bind address and port
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    -- INSERT --
```

```

        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    // Listen for incoming connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("Server listening on port %d...\n", PORT);
    // Accept a client connection
    if ((new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen)) < 0) {
        perror("Accept failed");
        exit(EXIT_FAILURE);
    }

    printf("Connection established with client.\n");
    // Communication loop
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(new_socket, buffer, BUFFER_SIZE);
        if (valread <= 0) {
            printf("Client disconnected.\n");
            break;
        }
        printf("Received: %s\n", buffer);
        // Capitalize the message
        capitalize(buffer);
        // Send the modified message back to the client
        send(new_socket, buffer, strlen(buffer), 0);
        printf("Capitalized & Sent: %s\n", buffer);
    }
    close(new_socket);
    close(server_fd);
    return 0;
}

```

```

hirushi@HS:~/Lab06$ vim 3s.c
hirushi@HS:~/Lab06$ gcc -o 3s 3s.c
hirushi@HS:~/Lab06$ ./3s
Server listening on port 3109...
Connection established with client.
Received: god help me please
Capitalized & Sent: GOD HELP ME PLEASE
Received: this is too hard
Capitalized & Sent: THIS IS TOO HARD

```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 3109
#define BUFFER_SIZE 1024
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};

    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }

    serv_addr.sin_family = AF_INET;    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and connect to server
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address / Address not supported");
        return -1;
    }

    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        return -1;
    }
    printf("Connected to server. Type messages below:\n");
    while (1) {
        printf("Client: ");
        fgets(buffer, BUFFER_SIZE, stdin);

        // Remove newline character if present
        -- INSERT --
```

```

// Convert IPv4 and connect to server
if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
    perror("Invalid address / Address not supported");
    return -1;
}

if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
    perror("Connection failed");
    return -1;
}
printf("Connected to server. Type messages below:\n");
while (1) {
    printf("Client: ");
    fgets(buffer, BUFFER_SIZE, stdin);

    // Remove newline character if present
    buffer[strcspn(buffer, "\n")] = 0;

    // Send message to server
    send(sock, buffer, strlen(buffer), 0);

    memset(buffer, 0, BUFFER_SIZE);

    // Read response from server
    int valread = read(sock, buffer, BUFFER_SIZE);
    if (valread <= 0) {
        printf("Server disconnected.\n");
        break;
    }
    printf("Server (capitalized): %s\n", buffer);
}
close(sock);
return 0;
}

```

```

hirushi@HS:~/Lab06$ vim 3c.c
hirushi@HS:~/Lab06$ rm 3c.c
hirushi@HS:~/Lab06$ vim 3c.c
hirushi@HS:~/Lab06$ gcc -o 3c 3c.c
hirushi@HS:~/Lab06$ ./3c
Connection failed: Connection refused
hirushi@HS:~/Lab06$ vim 3c.c
hirushi@HS:~/Lab06$ gcc -o 3c 3c.c
hirushi@HS:~/Lab06$ ./3c
Connected to server. Type messages below:
Client: god help me please
Server (capitalized): GOD HELP ME PLEASE
Client: this is too hard
Server (capitalized): THIS IS TOO HARD
Client: █

```

4)Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <arpa/inet.h>
#define PORT 7329
#define BUFFER_SIZE 128

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];

    // Create a socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }
    // Bind socket to address and port
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    // Listen for client connections
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
    printf("Time server listening on port %d...\n", PORT);
    // Accept client connection
    -- INSERT --
```

```

printf("Time server listening on port %d...\n", PORT);
// Accept client connection
if ((new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen)) < 0) {
    perror("Accept failed");
    exit(EXIT_FAILURE);
}
printf("Client connected.\n");
// Send the current time every second
while (1) {
    time_t rawtime;
    struct tm *timeinfo;
    // Get current time
    time(&rawtime);
    timeinfo = localtime(&rawtime);
    // Format time as a string
    strftime(buffer, BUFFER_SIZE, "%Y-%m-%d %H:%M:%S\n", timeinfo);
    // Send time to client
    send(new_socket, buffer, strlen(buffer), 0);
    printf("Sent time: %s", buffer);
    sleep(1); // Wait for 1 second
}
close(new_socket);
close(server_fd);
return 0;
}

```

```

hirushi@HS:~/Lab06$ rm 4s.c
hirushi@HS:~/Lab06$ vim 4s.c
hirushi@HS:~/Lab06$ gcc -o 4s 4s.c
hirushi@HS:~/Lab06$ ./4s
Time server listening on port 7329...
Client connected.
Sent time: 2025-02-08 22:55:28
Sent time: 2025-02-08 22:55:29
Sent time: 2025-02-08 22:55:30
Sent time: 2025-02-08 22:55:31
Sent time: 2025-02-08 22:55:32
Sent time: 2025-02-08 22:55:33
Sent time: 2025-02-08 22:55:34
Sent time: 2025-02-08 22:55:35
Sent time: 2025-02-08 22:55:36
Sent time: 2025-02-08 22:55:37
Sent time: 2025-02-08 22:55:38
Sent time: 2025-02-08 22:55:39
hirushi@HS:~/Lab06$ █

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 7329
#define BUFFER_SIZE 128
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation error");
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 address and connect to server
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address / Address not supported");
        return -1;
    }
    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        return -1;
    }
    printf("Connected to time server. Receiving time updates:\n");
    // Receive time from server continuously
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        int valread = read(sock, buffer, BUFFER_SIZE);
        if (valread <= 0) {
            printf("Server disconnected.\n");
            break;
        }
    }
}
-- INSERT --

```

5-23

```

        while (1) {
            memset(buffer, 0, BUFFER_SIZE);
            int valread = read(sock, buffer, BUFFER_SIZE);
            if (valread <= 0) {
                printf("Server disconnected.\n");
                break;
            }
            printf("Current Time: %s", buffer);
        }
    }
    close(sock);
    return 0;
}

```

```
hirushi@HS:~/Lab06$ vim 4s.c
hirushi@HS:~/Lab06$ vim 4c.c
hirushi@HS:~/Lab06$ gcc -o 4c 4c.c
hirushi@HS:~/Lab06$ ./4c
Connected to time server. Receiving time updates:
Current Time: 2025-02-08 22:55:28
Current Time: 2025-02-08 22:55:29
Current Time: 2025-02-08 22:55:30
Current Time: 2025-02-08 22:55:31
Current Time: 2025-02-08 22:55:32
Current Time: 2025-02-08 22:55:33
Current Time: 2025-02-08 22:55:34
  Current Time: 2025-02-08 22:55:35
Current Time: 2025-02-08 22:55:36
Current Time: 2025-02-08 22:55:37
Current Time: 2025-02-08 22:55:38
^C
```

(2)Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1000
#define FILE_NAME "serverfile.txt"

void send_file(int client_socket) {
    FILE *file = fopen(FILE_NAME, "rb");
    if (file == NULL) {
        perror("File not found");
        return;
    }

    // Hardcoded file size (change as needed)
    int file_size = 5000;
    send(client_socket, &file_size, sizeof(file_size), 0);

    char buffer[BUFFER_SIZE];
    int bytes_read;
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, file)) > 0) {
        send(client_socket, buffer, bytes_read, 0);
    }

    fclose(file);
    printf("File sent successfully.\n");
}

int main() {
    int server_fd, client_socket;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    -- INSERT --
```

```

socklen_t addr_len = sizeof(client_addr);

server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd == -1) {
    perror("Socket creation failed");
    exit(EXIT_FAILURE);
}

server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
    perror("Bind failed");
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 5) < 0) {
    perror("Listen failed");
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d...\n", PORT);
client_socket = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);
if (client_socket < 0) {
    perror("Accept failed");
    exit(EXIT_FAILURE);
}
char request[BUFFER_SIZE];
recv(client_socket, request, BUFFER_SIZE, 0);
if (strcmp(request, "request") == 0) {
    send_file(client_socket);
}
close(client_socket);
close(server_fd);
return 0;
}

```

```

hirushi@HS:~$ cd Lab06
hirushi@HS:~/Lab06$ vim e2_s.c
hirushi@HS:~/Lab06$ gcc -o e2_s e2_s.c
hirushi@HS:~/Lab06$ ./e2_s
Server listening on port 8080...
File sent successfully.
hirushi@HS:~/Lab06$

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1000
#define FILE_NAME "serverfile.txt"

int main() {
    int client_socket;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];
    FILE *file = fopen(FILE_NAME, "wb");

    if (file == NULL) {
        perror("File creation failed");
        exit(EXIT_FAILURE);
    }

    client_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (client_socket == -1) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;

    if (connect(client_socket, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }

    send(client_socket, "request", strlen("request") + 1, 0);
-- INSERT --

```

18.

```

    send(client_socket, "request", strlen("request") + 1, 0);

    int file_size;
    recv(client_socket, &file_size, sizeof(file_size), 0);
    printf("Receiving file of size: %d bytes\n", file_size);

    int bytes_received;
    while ((bytes_received = recv(client_socket, buffer, BUFFER_SIZE, 0)) > 0) {
        fwrite(buffer, 1, bytes_received, file);
    }

    fclose(file);
    close(client_socket);
    printf("File received successfully.\n");
    return 0;
}
-- INSERT --

```

```
hirushi@HS:~/Lab06$ vim e2_c.c
hirushi@HS:~/Lab06$ rm e2_c.c
hirushi@HS:~/Lab06$ vim e2_c.c
hirushi@HS:~/Lab06$ gcc -o e2_c e2_c.c
hirushi@HS:~/Lab06$ ./e2_c
Receiving file of size: 5000 bytes
File received successfully.
hirushi@HS:~/Lab06$ █
```